CENTRO UNIVERSITÁRIO FACENS
CURSO DE PÓS GRADUAÇÃO

REDES NEURAIS ARTIFICIAIS E

DEEP LEARNING / T2025/S2

APRESENTAÇÃO

NOSSAS AULAS SERÃO COM ENCONTROS REMOTOS NAS SEGUINTES DATAS:

17/01, 31/01, 21/02 e 07/03 NO HORÁRIO DAS 8:00 ÁS 17:15; TEREMOS UM PERIODO DE ALMOÇO DE UMA HORA E DOIS INTERVALOS DE 15 MINUTOS.

O CONTEÚDO DAS AULAS SERÃO GRAVADOS E O MATERIAL DISPONIBILIZADO PELO CANVAS;

TODOS OS EXERCÍCIOS SERÃO REALIZADOS DURANTE O TEMPO DOS QUATRO ENCONTROS;

TODAS AS ENTREGAS DEVEM SER FEITAS PELO CANVAS;

ALÉM DOS EXERCÍCIOS HAVERÁ UMA ENTREGA FINAL.

REVISÃO

SCHLEDER, Gabriel R.; FAZZIO, Adalberto. Machine Learning na Física, Química, e Ciência de Materiais: Descoberta e Design de Materiais. Revista Brasileira de Ensino de Física, v. 43, n. Suppl 1, p. e20200407, 2021.

REVISÃO

As Redes Neurais Artificiais (RNAs), também conhecidas como redes neurais ou ANN (Artificial Neural Networks, em inglês), são um modelo computacional e uma técnica de aprendizado de máquina inspirados na estrutura e no funcionamento do cérebro humano. Elas são usadas para resolver uma variedade de problemas complexos, incluindo classificação de imagens, reconhecimento de fala, previsão de séries temporais, tradução automática e muito mais.

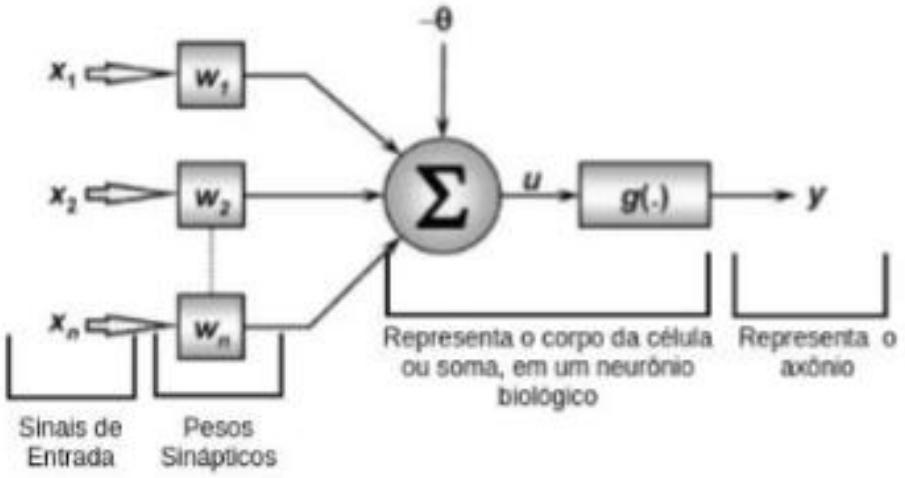
REVISÃO

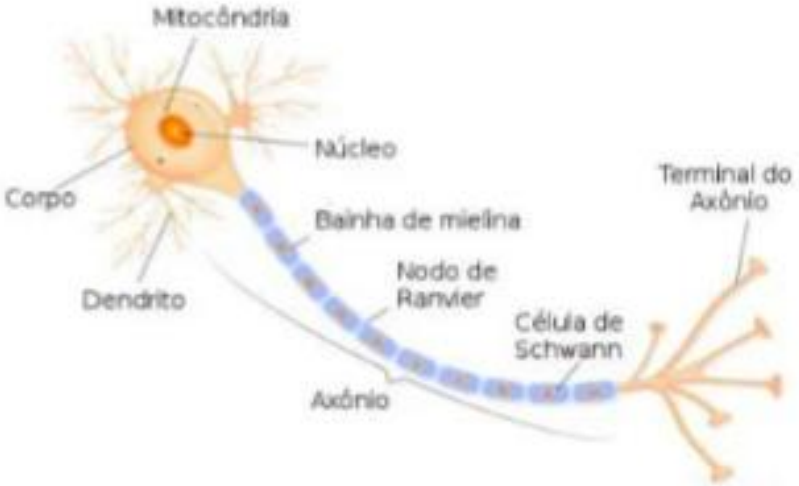
As RNAs consistem em unidades de processamento chamadas de "neurônios artificiais" ou "nós", organizadas em camadas interconectadas. **Existem três tipos principais de camadas em uma RNA:**

- **Camada de Entrada (Input Layer):** Esta camada recebe os dados de entrada e transmite-os para a camada seguinte. Cada neurônio na camada de entrada corresponde a uma característica ou atributo do dado de entrada.
- **Camadas Ocultas (Hidden Layers):** Estas são camadas intermediárias entre a camada de entrada e a camada de saída. Cada camada oculta contém vários neurônios que realizam cálculos complexos para aprender padrões nos dados.
- **Camada de Saída (Output Layer):** Esta camada fornece a resposta ou saída final da RNA com base nas informações processadas pelas camadas anteriores. O número de neurônios nesta camada depende do tipo de problema, como classificação (um neurônio para cada classe) ou regressão (um neurônio para um valor contínuo).

REVISÃO

As RNAs consistem em unidades de processamento chamadas de "neurônios artificiais" ou "nós", organizadas em camadas interconectadas.





https://www.researchgate.net/figure/Figura-19-Neuronio-biologico-esquerda-e-neuronio-artificial-direita_fig18_300015903

REVISÃO

Perceptron Simples (Single-Layer Perceptron):

Aplicações: Classificação linear de dados, separação de padrões lineares simples.

Redes Neurais Feedforward Multicamadas (Multilayer Perceptron - MLP):

Aplicações: Classificação, regressão, reconhecimento de padrões, processamento de linguagem natural, visão computacional, aprendizado profundo.

Redes Neurais Convolucionais (Convolutional Neural Networks - CNN):

Aplicações: Processamento de imagens, reconhecimento de objetos em imagens, classificação de imagens, visão computacional.

Redes Neurais Recorrentes (Recurrent Neural Networks - RNN):

Aplicações: Processamento de sequências temporais, análise de texto, tradução automática, previsão de séries temporais.

REVISÃO

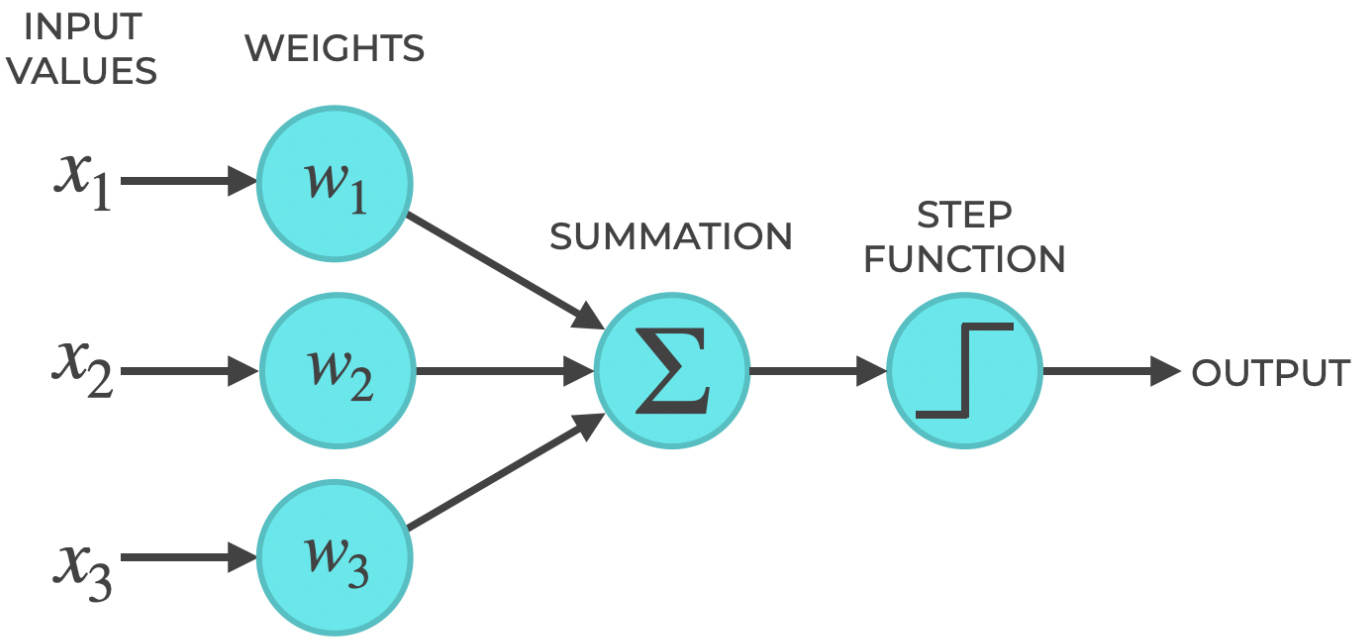
O **Perceptron** é um modelo de neurônio artificial específico que foi proposto por Frank Rosenblatt em 1957. É um modelo muito simples e pode ser visto como uma versão inicial e limitada dos neurônios artificiais. O Perceptron é projetado para resolver problemas de classificação linear, onde o objetivo é separar dados em duas classes com base em um limite de decisão linear. Ele usa uma função de ativação degrau (step function) para produzir uma saída binária (0 ou 1) com base na comparação da soma ponderada das entradas com um limite. Os Perceptrons não podem resolver problemas que não sejam linearmente separáveis, o que limita suas capacidades.

REVISÃO

```python
import numpy as np

# Função de ativação - Função degrau
def step_function(x):
    return 1 if x > 0 else 0

# Perceptron (neurônio artificial)
def perceptron(input_data, weights):
    # Calcula a soma ponderada das entradas e pesos
    weighted_sum = np.dot(input_data, weights)
    # Aplica a função de ativação
    output = step_function(weighted_sum)
    return output

# Pesos sinápticos (pesos do perceptron)
weights = np.array([0.5, 0.5])  # Inicialização aleatória

# Dados de entrada
input_data = np.array([0.2, 0.3])

# Calcula a saída do perceptron
output = perceptron(input_data, weights)

# Exibe a saída
print("Saída do perceptron:", output)
```

REVISÃO

THE STRUCTURE OF A PERCEPTRON



REVISÃO

Função de Ativação	Cálculo Matemático	Descrição	Sugestão de Uso
Step (Degrau)	$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$	Ativa o neurônio apenas quando a entrada ultrapassa um limiar (0 ou 1).	Perceptron Simples, problemas lineares binários.
Sigmoide	$f(x) = \frac{1}{1+e^{-x}}$	Suaviza a saída para um intervalo entre 0 e 1, interpretável como probabilidade.	Classificação binária, última camada em redes neurais para previsões probabilísticas.
Tanh (Tangente Hiperbólica)	$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	Similar à sigmoide, mas retorna valores entre -1 e 1, melhor para redes neurais profundas.	Camadas ocultas de redes neurais onde se deseja saídas centralizadas em torno de zero.
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$	Mantém apenas valores positivos, acelera o treinamento e reduz o problema do desvanecimento do gradiente.	Camadas ocultas em redes neurais profundas (CNNs, RNNs, etc.).
Leaky ReLU	$f(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}$	Similar à ReLU, mas permite pequenos valores negativos para evitar neurônios mortos.	Redes neurais profundas onde a ReLU convencional causa problemas com "neurônios mortos".
Softmax	$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	Converte valores em probabilidades para múltiplas classes, garantindo que a soma seja 1.	Última camada em problemas de classificação multiclasse.
Swish	$f(x) = x \cdot \text{sigmoid}(x)$	Aprimora a suavização do gradiente, permitindo valores negativos úteis.	Alternativa moderna ao ReLU em redes neurais profundas.

REVISÃO

1) Pré-processamento dos Dados:

Tratar valores faltantes (missing values).

Codificar variáveis categóricas (como gênero, tipo de contrato) em valores numéricos.

Normalizar as variáveis numéricas (como valor mensal) para garantir que estejam na mesma escala.

2) Divisão dos Dados:

Dividir a base em conjuntos de treinamento (70%) e teste (30%).

3) Implementação do Perceptron:

Criar um Perceptron Simples com uma função de ativação degrau ou sigmóide.

Treinar o modelo utilizando o algoritmo de descida do gradiente.

REVISÃO

4) Avaliação do Modelo:

Calcular métricas de desempenho, como acurácia, precisão, recall e F1-score.

Acurácia: Porcentagem de previsões corretas.

Precisão: Proporção de verdadeiros positivos entre todos os positivos previstos.

Recall: Proporção de verdadeiros positivos entre todos os positivos reais.

F1-Score: Média harmônica entre precisão e recall.

Analisar a matriz de confusão para entender os erros de classificação.

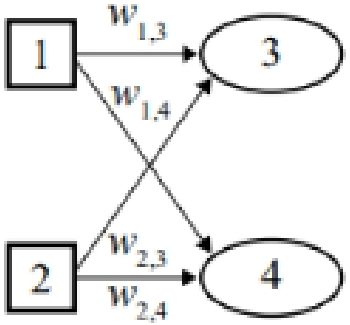
5) Interpretação dos Resultados:

Identificar quais variáveis têm maior impacto na previsão de churn.

Sugerir ações para a empresa com base nos clientes identificados como propensos ao churn.

REVISÃO

As redes neurais de camada única são uma estrutura fundamental com todas as suas entradas conectadas diretamente com somente uma saída, você pode ver uma rede de camada única simples de duas entradas com duas interligações.



x_1	x_2	$f(x)_1$	$f(x)_2$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

REVISÃO

As **redes multicamadas (MLP)** são uma extensão do **Perceptron**, compostas por várias camadas de neurônios interconectados. Essas camadas incluem:

- **Camada de entrada:** Recebe os dados brutos.
- **Camadas ocultas:** Realizam transformações não lineares nos dados.
- **Camada de saída:** Produz o resultado final.

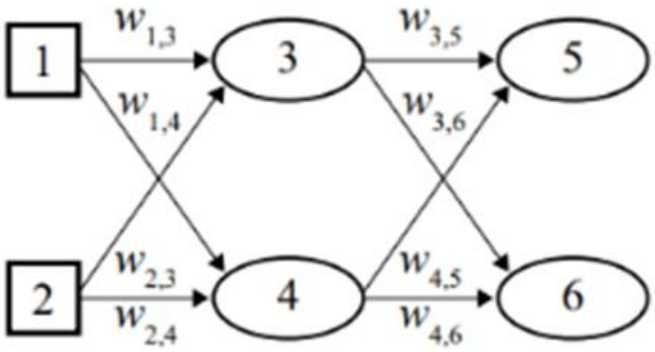
A principal vantagem das MLPs é sua capacidade de aprender representações hierárquicas dos dados, o que as torna poderosas para tarefas complexas.

REVISÃO

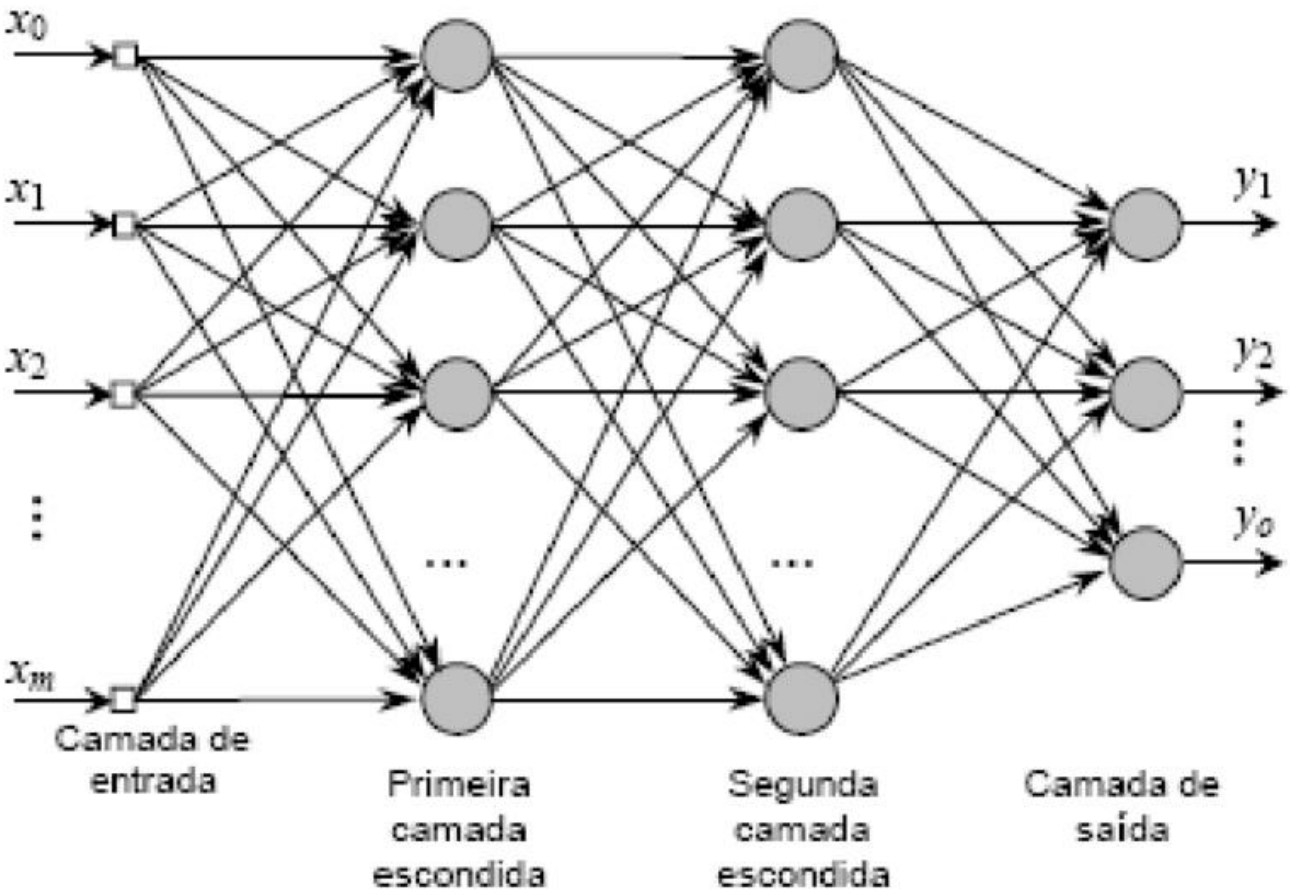
As **redes multicamadas (MLP)** são uma extensão do **Perceptron**, compostas por várias camadas de neurônios interconectados. Essas camadas incluem:

- **Camada de entrada:** Recebe os dados brutos.
- **Camadas ocultas:** Realizam transformações não lineares nos dados.
- **Camada de saída:** Produz o resultado final.

A principal vantagem das MLPs é sua capacidade de aprender representações hierárquicas dos dados, o que as torna poderosas para tarefas complexas.



REVISÃO


x_0
x_1
x_2
x_m
y_1
y_2
y_o
Camada de entrada
Primeira camada escondida
Segunda camada escondida
Camada de saída

PERCEPTRON

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Carregar os dados
df = pd.read_csv('Telco-Customer-Churn.csv')

# Selecionar as colunas relevantes
features = ['MonthlyCharges', 'Contract', 'OnlineSecurity', 'Churn']
df = df[features]

# Tratar valores faltantes
df = df.dropna()

# Codificação de variáveis categóricas
label_encoders = {}
for col in ['Contract', 'OnlineSecurity', 'Churn']:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Separar variáveis independentes e dependente
X = df[['MonthlyCharges', 'Contract', 'OnlineSecurity']].values
y = df['Churn'].values

# Normalizar os dados
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Divisão entre treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

```

```python

# Implementação do Perceptron
class Perceptron:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.weights = None
        self.bias = None

    def activation(self, x):
        return 1 if x >= 0 else 0

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for _ in range(self.epochs):
            for idx, x_i in enumerate(X):
                linear_output = np.dot(x_i, self.weights) + self.bias
                y_pred = self.activation(linear_output)

                # Atualização dos pesos
                update = self.lr * (y[idx] - y_pred)
                self.weights += update * x_i
                self.bias += update

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return np.array([self.activation(x) for x in linear_output])

# Treinar o modelo
perceptron = Perceptron(lr=0.01, epochs=1000)
perceptron.fit(X_train, y_train)

y_pred = perceptron.predict(X_test)

```

```python
# Avaliação do modelo
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"Acurácia: {accuracy:.4f}")
print("Matriz de Confusão:\n", conf_matrix)
print("Relatório de Classificação:\n", report)

```

```
Acurácia: 0.7132
Matriz de Confusão:
 [[1342  197]
 [ 409  165]]
Relatório de Classificação:
               precision    recall  f1-score   support

           0       0.77      0.87      0.82      1539
           1       0.46      0.29      0.35       574

    accuracy                           0.71      2113
   macro avg       0.61      0.58      0.58      2113
weighted avg       0.68      0.71      0.69      2113
```

O resultado mostra que a acurácia do modelo foi de 71,32%, mas há um grande desequilíbrio na capacidade do Perceptron de prever corretamente a classe "1" (clientes que cancelam o serviço). O recall para essa classe está muito baixo (0.29), indicando que o modelo está deixando de prever corretamente muitos clientes propensos ao churn.

REDE NEURAL DE CAMADA ÚNICA

```python
import numpy as np
from sklearn.preprocessing import StandardScaler

# Função de ativação Softmax
def softmax(x):
    exp_x = np.exp(x - np.max(x))  # Subtrai o máximo para evitar overflow numérico
    return exp_x / exp_x.sum(axis=0)

# Treinamento do perceptron para múltiplas classes
def treinar_perceptron(entradas, saidas, taxa_aprendizagem, epocas):
    num_classes = len(set(saidas))  # Determina a quantidade de classes
    pesos = np.random.rand(entradas.shape[1], num_classes)

    for epoca in range(epocas):
        for i in range(len(entradas)):
            # Calcular soma ponderada
            soma_ponderada = np.dot(entradas[i], pesos)

            # Calcular saída do perceptron com softmax
            probabilidades = softmax(soma_ponderada)

            # Criar vetor esperado de saída one-hot
            y_real = np.zeros(num_classes)
            y_real[saidas[i]] = 1

            # Ajustar pesos com base no erro
            erro = y_real - probabilidades
            pesos += taxa_aprendizagem * np.outer(entradas[i], erro)

    return pesos
```

```python

# Exemplo de uso
dados_entrada = np.array([
    [5.1, 3.5, 1.4],  # Íris
    [4.9, 3.0, 1.4],  # Íris
    [5.1, 3.6, 1.4],  # Íris
    [6.4, 3.0, 1.7],  # Lírio
    [6.3, 3.3, 1.5],  # Lírio
    [5.7, 3.0, 1.3],  # Rosa
    [5.8, 2.7, 1.3],  # Rosa
])

rotulos_saida = np.array([0, 0, 0, 1, 1, 2, 2])

# Normalizar os dados
scaler = StandardScaler()
dados_entrada = scaler.fit_transform(dados_entrada)

taxa_aprendizagem = 0.1
epocas = 300

# Treinar o perceptron
pesos_treinados = treinar_perceptron(dados_entrada, rotulos_saida, taxa_aprendizagem, epocas)

# Classificar nova flor
nova_flor = np.array([[5.8, 2.5, 1.0]])
nova_flor = scaler.transform(nova_flor)
soma_ponderada = np.dot(nova_flor, pesos_treinados)
probabilidades = softmax(soma_ponderada[0])
saida = np.argmax(probabilidades)

if saida == 0:
    print("Nova flor classificada como Íris")
elif saida == 1:
    print("Nova flor classificada como Lírio")
else:
    print("Nova flor classificada como Rosa")
```

REDE NEURAL MULTIPLAS CAMADAS

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Carregar o dataset de dígitos
digits = load_digits()
X = digits.data
y = digits.target

# Dividir os dados em conjuntos de treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalizar os dados
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Criar e treinar a MLP
mlp = MLPClassifier(hidden_layer_sizes=(50,), max_iter=1000, random_state=42)
mlp.fit(X_train, y_train)

# Fazer previsões no conjunto de teste
y_pred = mlp.predict(X_test)
```

```python
# Fazer previsões no conjunto de teste
y_pred = mlp.predict(X_test)

# Avaliar o modelo
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Visualizar alguns resultados
def plot_digits(X, y_true, y_pred=None):
    fig, axes = plt.subplots(4, 4, figsize=(10, 10))
    for i, ax in enumerate(axes.flat):
        ax.imshow(X[i].reshape(8, 8), cmap='gray')
        if y_pred is not None:
            ax.set_title(f'True: {y_true[i]}\nPred: {y_pred[i]}')
        else:
            ax.set_title(f'True: {y_true[i]}')
        ax.axis('off')
    plt.show()

# Mostrar alguns dígitos com suas previsões
plot_digits(X_test, y_test, y_pred)
```

DÚVIDAS ou PERGUNTAS?

MUITO OBRIGADO!!!!

daniel.ohata@facens.br

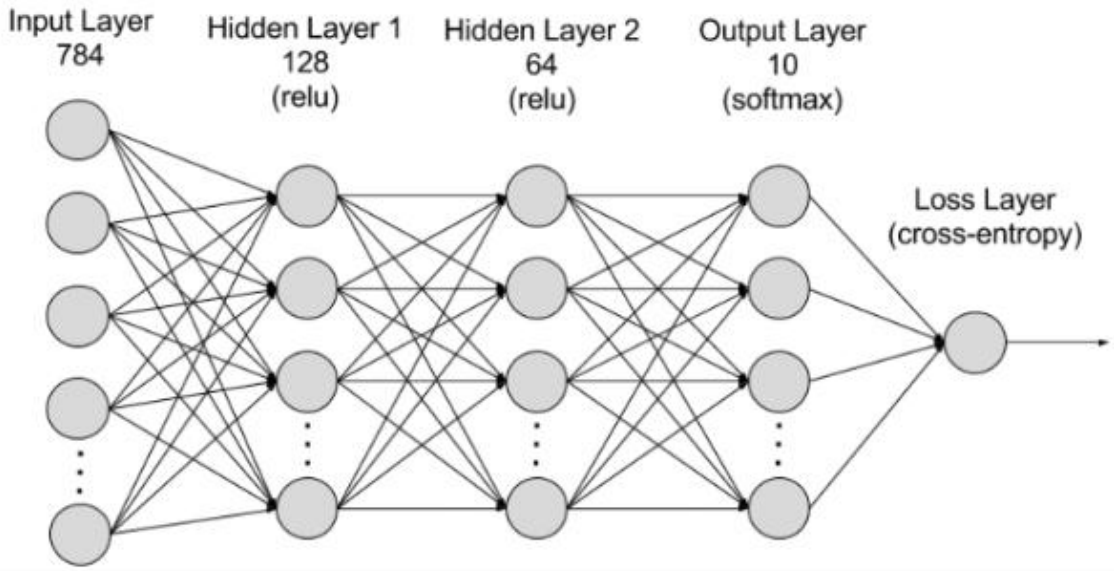
DEEP LEARNING

Deep Learning (Aprendizado Profundo) é uma subcategoria do Machine Learning que utiliza redes neurais artificiais profundas para aprender e realizar tarefas complexas de reconhecimento de padrões. Alguns exemplos de aplicação:

- Reconhecimento de Imagens;
- Processamento de Linguagem Natural;
- Recomendações Personalizadas;
- Carros Autônomos;
- Reconhecimento de Voz;
- Jogos e Estratégia;
- Processamento de Sinais.

DEEP LEARNING

Uma rede neural é um método de inteligência artificial que ensina computadores a processar dados de uma forma inspirada pelo cérebro humano. É um tipo de processo de machine learning, chamado aprendizado profundo, que usa nós ou neurônios interconectados em uma estrutura em camadas, semelhante ao cérebro humano.



https://aws.amazon.com/pt/what-is/neural-network/#:~:text=Uma%20rede%20neural%20%C3%A9%20um,camadas%2C%20semelhante%20ao%20c%C3%A9rebro%20humano.

DEEP LEARNING

O neurônio artificial é a unidade matemática básica das redes neurais. Ele recebe entradas, aplica pesos, soma os valores e utiliza uma função de ativação para produzir uma saída.

O perceptron é um modelo de aprendizado supervisionado que utiliza um neurônio artificial com uma função de ativação de limiar (step) para realizar classificação binária linear. Ele também possui um algoritmo de treinamento para ajustar os pesos.

Em outras palavras:

neurônio artificial → componente matemático

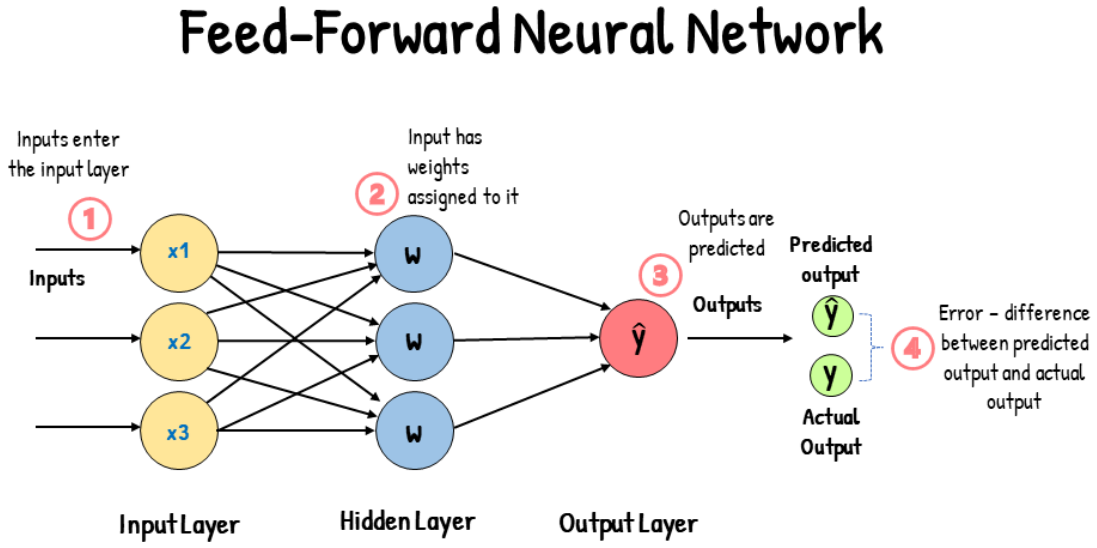
perceptron → algoritmo de classificação que utiliza esse componente

DEEP LEARNING

Critério	Neurônio Artificial	Perceptron
Natureza	Unidade matemática básica de redes neurais	Modelo de aprendizado supervisionado
Função	Realiza soma ponderada das entradas e aplica função de ativação	Usa um neurônio artificial para classificar dados
Estrutura	Entrada + pesos + bias + função de ativação	Mesmo neurônio artificial + regra de aprendizado
Tipo de problema	Usado em vários modelos de redes neurais	Classificação binária
Função de ativação	Pode usar várias (ReLU, Sigmoid, Tanh, Softmax)	Geralmente função degrau (step)
Treinamento	Não define sozinho um método de aprendizado	Possui regra de atualização de pesos
Capacidade de modelagem	Depende da rede onde está inserido	Resolve apenas problemas linearmente separáveis
Uso em Deep Learning	Base de todas as redes neurais modernas	Considerado um modelo inicial das redes neurais
Papel no modelo	Componente da arquitetura	Algoritmo completo de classificação

DEEP LEARNING

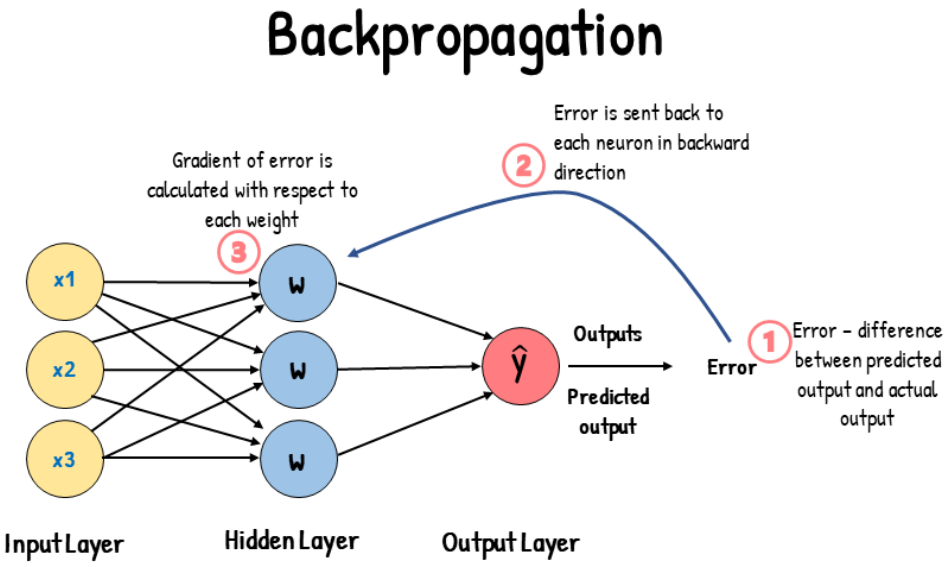
O feedforward, também conhecido como propagação direta, é o processo pelo qual os dados são propagados através de uma rede neural, camada por camada, da entrada até a saída. Durante o feedforward, cada camada da rede neural realiza duas operações principais: a multiplicação dos dados de entrada pelos pesos das conexões sinápticas e a aplicação de uma função de ativação.



https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/

DEEP LEARNING

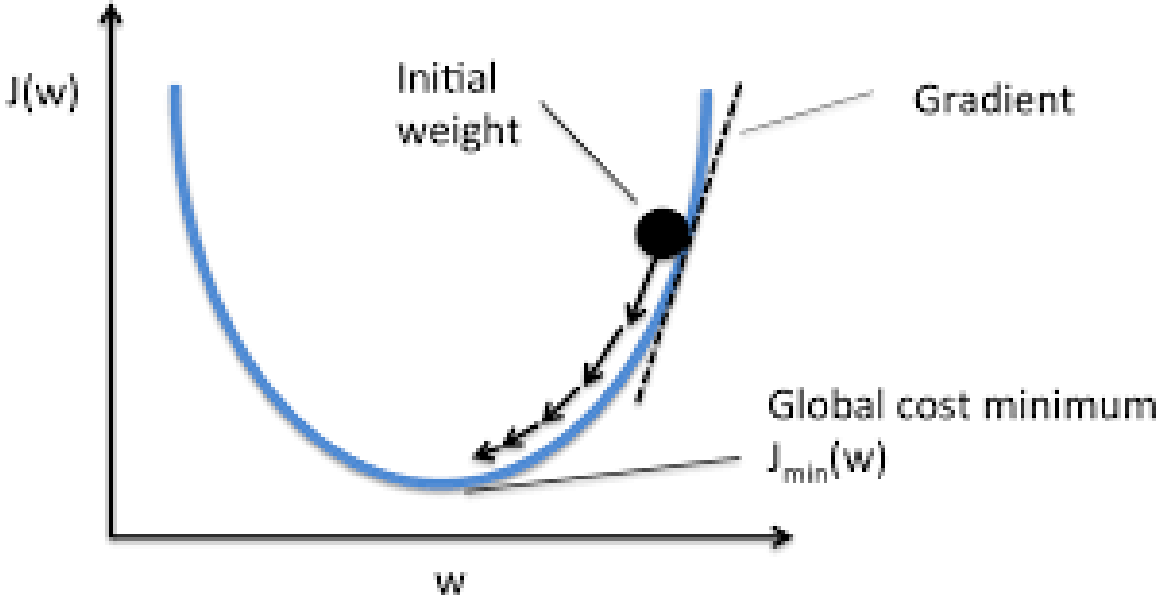
O backpropagation é um algoritmo de treinamento iterativo que calcula os gradientes da função de perda em relação aos pesos da rede neural, retropropaga esses gradientes através da rede para calcular os gradientes para cada camada oculta e, finalmente, usa esses gradientes para atualizar os pesos da rede, ajustando-os de forma que a função de perda seja minimizada.



https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/

DEEP LEARNING

A descida de gradiente estocástica é uma técnica de otimização eficiente que permite treinar modelos de aprendizado de máquina, incluindo redes neurais, usando amostragem estocástica para calcular os gradientes da função de perda e fazer atualizações de peso mais frequentes durante o treinamento.



https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/

DEEP LEARNING

O objetivo do código a seguir é treinar uma rede neural simples para realizar a operação XOR. O XOR é um problema de classificação binária onde a saída é 1 se apenas uma das entradas é 1, e 0 caso contrário.

Este exemplo demonstra como implementar uma rede neural multicamadas do zero usando NumPy, aplicando conceitos como feedforward, backpropagation e descida de gradiente estocástica (SGD). O mesmo princípio pode ser aplicado para resolver outros problemas de classificação ou regressão.

DEEP LEARNING

```python
import numpy as np

# Função de ativação sigmoid
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Função de perda (erro quadrático médio)
def mse_loss(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

# Dados de entrada e saída
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Inicialização dos pesos e viés (bias) de forma aleatória
np.random.seed(42)
input_dim = X.shape[1]
hidden_dim = 3
output_dim = 1
W1 = np.random.randn(input_dim, hidden_dim)
b1 = np.zeros((1, hidden_dim))
W2 = np.random.randn(hidden_dim, output_dim)
b2 = np.zeros((1, output_dim))

# Hiperparâmetros
learning_rate = 0.1
epochs = 10000
```

```python
# Treinamento da rede neural
for epoch in range(epochs):
    # Feedforward
    z1 = np.dot(X, W1) + b1
    a1 = sigmoid(z1)
    z2 = np.dot(a1, W2) + b2
    y_pred = sigmoid(z2)

    # Backpropagation
    error = mse_loss(y, y_pred)
    if epoch % 1000 == 0:
        print(f"Epoch {epoch}: Loss = {error}")

    # Cálculo do gradiente
    delta2 = (y_pred - y) * y_pred * (1 - y_pred)
    dW2 = np.dot(a1.T, delta2)
    db2 = np.sum(delta2, axis=0, keepdims=True)
    delta1 = np.dot(delta2, W2.T) * a1 * (1 - a1)
    dW1 = np.dot(X.T, delta1)
    db1 = np.sum(delta1, axis=0, keepdims=True)

    # Atualização dos pesos e viés (bias)
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

# Teste da rede neural treinada
print("\nTeste da rede neural:")
for i in range(len(X)):
    z1 = np.dot(X[i], W1) + b1
    a1 = sigmoid(z1)
    z2 = np.dot(a1, W2) + b2
    y_pred = sigmoid(z2)
    print(f"Entrada: {X[i]} - Saída prevista: {y_pred}")
```

DEEP LEARNING

O Keras é um framework flexível para construção e treinamento de redes neurais em Python, adequado para uma ampla gama de aplicações de aprendizado de máquina e deep learning. Ele simplifica o processo de desenvolvimento de modelos de redes neurais, permitindo que os desenvolvedores foquem na criação e experimentação de arquiteturas de rede, em vez de lidar com detalhes de implementação complexos.

DEEP LEARNING

```python
import numpy as np
from keras.models import Sequential
from keras.layers import Dense

# Dados de entrada e saída
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Construção do modelo
model = Sequential()
model.add(Dense(3, input_dim=2, activation='sigmoid'))  # Camada oculta com 3 neurônios e função de ativação sigmoid
model.add(Dense(1, activation='sigmoid'))  # Camada de saída com 1 neurônio e função de ativação sigmoid

# Compilação do modelo
model.compile(loss='mean_squared_error', optimizer='sgd')  # Configuração da função de perda e otimizador

# Treinamento do modelo
model.fit(X, y, epochs=10000, batch_size=1, verbose=0)  # Treinamento com 10000 épocas e tamanho do lote 1

# Avaliação do modelo
print("\nTeste da rede neural:")
for i in range(len(X)):
    y_pred = model.predict(np.array([X[i]]))[0][0]
    print(f"Entrada: {X[i]} - Saída prevista: {y_pred}")
```

DEEP LEARNING

O TensorFlow é uma biblioteca de código aberto desenvolvida pela equipe do Google Brain para computação numérica e programação de modelos de aprendizado de máquina, incluindo redes neurais profundas. Ele foi lançado em 2015 e desde então se tornou uma das principais ferramentas para desenvolvimento e implantação de modelos de aprendizado de máquina em larga escala.

TensorFlow

DEEP LEARNING

```python
import numpy as np
import tensorflow as tf

# Dados de entrada e saída
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Construção do modelo
model = tf.keras.Sequential([
    tf.keras.layers.Dense(3, input_dim=2, activation='sigmoid'),  # Camada oculta com 3 neurônios e função de ativação sigmoid
    tf.keras.layers.Dense(1, activation='sigmoid')  # Camada de saída com 1 neurônio e função de ativação sigmoid
])

# Compilação do modelo
model.compile(loss='mean_squared_error', optimizer='sgd')  # Configuração da função de perda e otimizador

# Treinamento do modelo
model.fit(X, y, epochs=10000, batch_size=1, verbose=0)  # Treinamento com 10000 épocas e tamanho do lote 1

# Avaliação do modelo
print("\nTeste da rede neural:")
for i in range(len(X)):
    y_pred = model.predict(np.array([X[i]]))[0][0]
    print(f"Entrada: {X[i]} - Saída prevista: {y_pred}")
```

DEEP LEARNING

Redes Neurais Convolucionais (CNNs): São redes neurais especializadas para processamento de imagens. Elas têm uma arquitetura que explora a estrutura espacial dos dados, fazendo uso de camadas convolucionais, de pooling e totalmente conectadas para extrair características relevantes das imagens.

Redes Neurais Recorrentes (RNNs): São redes neurais projetadas para lidar com dados sequenciais, como texto, áudio e séries temporais. Elas possuem uma estrutura que permite que informações sejam mantidas ao longo do tempo, permitindo a modelagem de dependências temporais complexas.

Redes Generativas Adversariais (GANs): São compostas por dois modelos, um gerador e um discriminador, que são treinados de forma adversarial. O gerador cria novas amostras que são indistinguíveis das amostras reais, enquanto o discriminador tenta distinguir entre amostras reais e falsas.

DEEP LEARNING

Redes Neurais Convolucionais (CNNs): São redes neurais especializadas para processamento de imagens. Elas têm uma arquitetura que explora a estrutura espacial dos dados, fazendo uso de camadas convolucionais, de pooling e totalmente conectadas para extrair características relevantes das imagens.

Redes Neurais Recorrentes (RNNs): São redes neurais projetadas para lidar com dados sequenciais, como texto, áudio e séries temporais. Elas possuem uma estrutura que permite que informações sejam mantidas ao longo do tempo, permitindo a modelagem de dependências temporais complexas.

Redes Generativas Adversariais (GANs): São compostas por dois modelos, um gerador e um discriminador, que são treinados de forma adversarial. O gerador cria novas amostras que são indistinguíveis das amostras reais, enquanto o discriminador tenta distinguir entre amostras reais e falsas.

DEEP LEARNING

O conjunto de dados MNIST é um conjunto padrão amplamente utilizado na comunidade de aprendizado de máquina e visão computacional. Ele consiste em um conjunto de 70.000 imagens em escala de cinza de dígitos manuscritos, cada uma com tamanho de 28x28 pixels. Essas imagens estão divididas em um conjunto de treinamento com 60.000 exemplos e um conjunto de teste com 10.000 exemplos.

Cada imagem no conjunto de dados MNIST mostra um único dígito manuscrito (0 a 9) centralizado na imagem. O objetivo típico ao trabalhar com o MNIST é desenvolver algoritmos capazes de identificar corretamente qual dígito é mostrado em cada imagem. Isso é uma tarefa de classificação, onde o modelo precisa atribuir a cada imagem uma classe correspondente ao dígito que ela representa.

https://www.tensorflow.org/datasets/catalog/mnist?hl=pt-br

REDES NEURAIS CONVOLUCIONAIS (CNNs):

```python
# Importação das bibliotecas necessárias
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Carregamento do conjunto de dados MNIST (já incluso no Keras)
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()

# Pré-processamento dos dados
train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images / 255.0

test_images = test_images.reshape((10000, 28, 28, 1))
test_images = test_images / 255.0

# Criação do modelo da rede neural convolucional
model = tf.keras.models.Sequential([
    # Camada convolucional com 32 filtros de 3x3
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),

    # Camada de pooling com tamanho de janela 2x2
    tf.keras.layers.MaxPooling2D((2, 2)),

    # Camada convolucional com 64 filtros de 3x3
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),

    # Camada de pooling com tamanho de janela 2x2
    tf.keras.layers.MaxPooling2D((2, 2)),

    # Camada de achatamento para converter os dados em um vetor unidimensional
    tf.keras.layers.Flatten(),
```

REDES NEURAIS CONVOLUCIONAIS (CNNs):

```
    # Camada totalmente conectada com 128 neurônios e função de ativação relu
    tf.keras.layers.Dense(128, activation='relu'),

    # Camada de saída com 10 neurônios e função de ativação softmax
    tf.keras.layers.Dense(10, activation='softmax')
])

# Compilação do modelo
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Treinamento do modelo
model.fit(train_images, train_labels, epochs=5)
```

REDES NEURAIS CONVOLUCIONAIS (CNNs):

Redes Neurais Recorrentes (RNNs) são um tipo de rede neural artificial projetada para lidar com dados sequenciais ou temporais. Elas são caracterizadas por terem conexões de retroalimentação, o que significa que informações podem ser transmitidas de um passo de tempo para outro dentro da rede.

A principal vantagem das RNNs é a capacidade de lidar com entradas de comprimento variável, o que as torna úteis para uma variedade de tarefas, como processamento de linguagem natural (NLP), reconhecimento de voz, previsão de séries temporais, entre outras.

O mecanismo de retroalimentação permite que as RNNs mantenham uma espécie de memória interna, o que lhes permite capturar padrões sequenciais complexos. No entanto, as RNNs tradicionais podem sofrer de problemas de dissipação ou explosão de gradiente durante o treinamento, o que pode dificultar a aprendizagem de dependências de longo prazo.

REDES NEURAIS RECORRENTES (RNNs):

```python
import numpy as np
import tensorflow as tf

# Dados de exemplo
text = "a quick brown fox jumps over the lazy dog"
words = text.split()
word_to_id = {word: idx for idx, word in enumerate(set(words))}
id_to_word = {idx: word for word, idx in word_to_id.items()}
vocab_size = len(word_to_id)

# Convertendo palavras para IDs
word_ids = [word_to_id[word] for word in words]

# Criando sequências de entrada e saída
seq_length = 3
x_data = []
y_data = []
for i in range(0, len(word_ids) - seq_length):
    x_data.append(word_ids[i:i+seq_length])
    y_data.append(word_ids[i+seq_length])

# Convertendo para numpy arrays
x_data = np.array(x_data)
y_data = np.array(y_data)

# Parâmetros da rede
num_epochs = 10
batch_size = 16
learning_rate = 0.01
hidden_size = 10
```

REDES NEURAIS RECORRENTES (RNNs):

```python
# Construindo o modelo
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, hidden_size, input_length=seq_length),
    tf.keras.layers.LSTM(hidden_size),
    tf.keras.layers.Dense(vocab_size, activation='softmax')
])

# Compilando o modelo
model.compile(loss='sparse_categorical_crossentropy', optimizer=tf.keras.optimizers.Adam(learning_rate))

# Treinando o modelo
model.fit(x_data, y_data, epochs=num_epochs, batch_size=batch_size, verbose=2)

# Função para gerar texto
def generate_text(seed_text, num_words):
    seed_ids = [word_to_id[word] for word in seed_text.split()]
    for _ in range(num_words):
        padded_seed = np.array([seed_ids + [0] * (seq_length - len(seed_ids))])
        predicted_id = np.argmax(model.predict(padded_seed), axis=-1)
        seed_ids.append(predicted_id[0])
    generated_text = ' '.join([id_to_word[idx] for idx in seed_ids])
    return generated_text

# Gerando texto de exemplo
generated_text = generate_text("quick brown", 5)
print("Generated Text:", generated_text)
```

REDES NEURAIS RECORRENTES (RNNs):

Redes Neurais Recorrentes (RNNs) são um tipo de rede neural artificial projetada para lidar com dados sequenciais ou temporais. Elas são caracterizadas por terem conexões de retroalimentação, o que significa que informações podem ser transmitidas de um passo de tempo para outro dentro da rede.

A principal vantagem das RNNs é a capacidade de lidar com entradas de comprimento variável, o que as torna úteis para uma variedade de tarefas, como processamento de linguagem natural (NLP), reconhecimento de voz, previsão de séries temporais, entre outras.

O mecanismo de retroalimentação permite que as RNNs mantenham uma espécie de memória interna, o que lhes permite capturar padrões sequenciais complexos. No entanto, as RNNs tradicionais podem sofrer de problemas de dissipação ou explosão de gradiente durante o treinamento, o que pode dificultar a aprendizagem de dependências de longo prazo.

REDES NEURAIS RECORRENTES (RNNs):

```python
import numpy as np
import tensorflow as tf

# Dados de exemplo
text = "a quick brown fox jumps over the lazy dog"
words = text.split()
word_to_id = {word: idx for idx, word in enumerate(set(words))}
id_to_word = {idx: word for word, idx in word_to_id.items()}
vocab_size = len(word_to_id)

# Convertendo palavras para IDs
word_ids = [word_to_id[word] for word in words]

# Criando sequências de entrada e saída
seq_length = 3
x_data = []
y_data = []
for i in range(0, len(word_ids) - seq_length):
    x_data.append(word_ids[i:i+seq_length])
    y_data.append(word_ids[i+seq_length])

# Convertendo para numpy arrays
x_data = np.array(x_data)
y_data = np.array(y_data)

# Parâmetros da rede
num_epochs = 10
batch_size = 16
learning_rate = 0.01
hidden_size = 10
```

REDES NEURAIS RECORRENTES (RNNs):

```python
# Construindo o modelo
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, hidden_size, input_length=seq_length),
    tf.keras.layers.LSTM(hidden_size),
    tf.keras.layers.Dense(vocab_size, activation='softmax')
])

# Compilando o modelo
model.compile(loss='sparse_categorical_crossentropy', optimizer=tf.keras.optimizers.Adam(learning_rate))

# Treinando o modelo
model.fit(x_data, y_data, epochs=num_epochs, batch_size=batch_size, verbose=2)

# Função para gerar texto
def generate_text(seed_text, num_words):
    seed_ids = [word_to_id[word] for word in seed_text.split()]
    for _ in range(num_words):
        padded_seed = np.array([seed_ids + [0] * (seq_length - len(seed_ids))])
        predicted_id = np.argmax(model.predict(padded_seed), axis=-1)
        seed_ids.append(predicted_id[0])
    generated_text = ' '.join([id_to_word[idx] for idx in seed_ids])
    return generated_text

# Gerando texto de exemplo
generated_text = generate_text("quick brown", 5)
print("Generated Text:", generated_text)
```

REDES NEURAIS GENERATIVAS ADVERSARIAIS (GANs):

Redes Neurais Generativas Adversariais (GANs) são um tipo de arquitetura de rede neural utilizada em aprendizado de máquina não supervisionado, proposta por Ian Goodfellow e seus colegas em 2014. A principal ideia por trás das GANs é treinar duas redes neurais simultaneamente: o gerador e o discriminador.

Gerador: O gerador cria novas amostras de dados, como imagens, a partir de um espaço de entrada latente (geralmente, um vetor de números aleatórios). O objetivo do gerador é criar amostras que sejam indistinguíveis das reais.

Discriminador: O discriminador recebe amostras do gerador e amostras reais. Ele é treinado para distinguir entre as amostras geradas e as reais. Ou seja, o discriminador tenta classificar corretamente se uma amostra é real ou foi gerada artificialmente.

REDES NEURAIS GENERATIVAS ADVERSARIAIS (GANs):

```python
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# Definir o gerador
def build_generator():
    model = models.Sequential()
    model.add(layers.Dense(128, activation='relu', input_shape=(100,)))
    model.add(layers.Dense(784, activation='sigmoid'))
    model.add(layers.Reshape((28, 28, 1)))
    return model

# Definir o discriminador
def build_discriminator():
    model = models.Sequential()
    model.add(layers.Flatten(input_shape=(28, 28, 1)))
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

# Função para treinar o discriminador
def train_discriminator(discriminator, real_images, fake_images):
    real_labels = np.ones((real_images.shape[0], 1))
    fake_labels = np.zeros((fake_images.shape[0], 1))

    real_loss = discriminator.train_on_batch(real_images, real_labels)
    fake_loss = discriminator.train_on_batch(fake_images, fake_labels)

    return real_loss, fake_loss
```

```python
# Função para treinar a GAN
def train_gan(generator, discriminator, gan, batch_size, latent_dim):
    fake_latent = np.random.randn(batch_size, latent_dim)
    fake_images = generator.predict(fake_latent)

    gan_labels = np.ones((batch_size, 1))

    gan_loss = gan.train_on_batch(fake_latent, gan_labels)

    return gan_loss

# Função para visualizar imagens geradas
def visualize_generated_images(generator, latent_dim, num_samples=10):
    noise = np.random.randn(num_samples, latent_dim)
    generated_images = generator.predict(noise)

    plt.figure(figsize=(10, 10))
    for i in range(num_samples):
        plt.subplot(1, num_samples, i+1)
        plt.imshow(generated_images[i, :, :, 0], cmap='gray')
        plt.axis('off')
    plt.show()

# Parâmetros
latent_dim = 100
batch_size = 128
epochs = 10000

# Construir e compilar os modelos
generator = build_generator()
generator.compile(loss='binary_crossentropy', optimizer='adam')

discriminator = build_discriminator()
discriminator.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```python
discriminator.trainable = False
gan_input = tf.keras.Input(shape=(latent_dim,))
gan_output = discriminator(generator(gan_input))
gan = tf.keras.Model(gan_input, gan_output)
gan.compile(loss='binary_crossentropy', optimizer='adam')

# Carregar dados reais (por exemplo, do MNIST)
(train_images, _), (_, _) = tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).astype('float32')
train_images = (train_images - 127.5) / 127.5

# Treinamento da GAN
for epoch in range(epochs):
    idx = np.random.randint(0, train_images.shape[0], batch_size)
    real_images = train_images[idx]

    fake_latent = np.random.randn(batch_size, latent_dim)
    fake_images = generator.predict(fake_latent)

    d_loss_real, d_loss_fake = train_discriminator(discriminator, real_images, fake_images)

    gan_loss = train_gan(generator, discriminator, gan, batch_size, latent_dim)

    if epoch % 100 == 0:
        print(f"Epoch: {epoch}, D Loss Real: {d_loss_real}, D Loss Fake: {d_loss_fake}, GAN Loss: {gan_loss}")
        visualize_generated_images(generator, latent_dim)
```

EXERCÍCIO

EXERCÍCIO 1

- PARA A TERCEIRA ETAPA, FAÇA O DOWNLOAD DO MATERIAL DA TERCEIRA AULA;
- VEJA CADA UM DOS EXEMPLOS PRÁTICOS SOBRE OS MODELOS TRADICIONAIS DE DEEP LEARNING:
- REDE NEURAL (SEM OS FRAMEWORKS); REDE NEURAL (COM KERAS); REDE NEURAL (TENSORFLOW); REDES NEURAIS CONVOLUCIONAIS; REDES NEURAIS RECORRENTES ;REDE NEURAIS GENERATIVAS ADVERSARIAIS
- ABRA SEU GOOGLE COLAB E NO MESMO NOTEBOOK COMPARTILHADO, REESCREVA OS CÓDIGOS DE EXEMPLO;
- A CADA CÓDIGO ESCRITO SEPARE COM UM TÍTULO DO CÓDIGO;
- AO TÉRMINO COMPILE CADA CÓDIGO E VERIFIQUE A INFORMAÇÃO DE SAÍDA OBTIDA.
- ESSA ATIVIDADE É INDIVIDUAL E SERÁ ENTREGUE PELO COMPARTILHAMENTO DO NOTEBOOK PELO E-MAIL DANIEL.OHATA@FACENS.BR
- CERTIFIQUE SE O PROFESSOR TEM ACESSO AO SEU NOTEBOOK.

PROBLEMAS DAS RNN(REDES NEURAIS RECORRENTES)

Em uma RNN simples, a informação é passada adiante através de um único estado oculto que é sobrescrito e transformado a cada passo. À medida que a distância entre a informação relevante e o ponto onde ela é necessária aumenta, a RNN tem dificuldade em conectar esses dois pontos. O sinal dessa informação original vai se "diluindo" ou sendo "corrompido" pela mistura com todas as outras informações intermediárias, tornando-se irreconhecível.

Para resolver o problema do gradiente desaparecente e permitir o aprendizado de dependências longas, foram desenvolvidas arquiteturas que controlam explicitamente o fluxo de informação através de um mecanismo de **portões (gates)**.

PROBLEMAS DAS RNN(REDES NEURAIS RECORRENTES)

A ideia central é criar um caminho de dados que seja preservado ao longo do tempo. Em vez de ter um único estado oculto que é totalmente sobrescrito, essas arquiteturas possuem uma "célula de memória" e mecanismos que decidem:

- **O que esquecer** da memória antiga.
- **O que guardar** da nova informação.
- **O que usar** para a saída no momento atual.

Esses portões são camadas neurais (geralmente com ativação sigmoide, que gera valores entre 0 e 1) que aprendem, durante o treinamento, quando abrir ou fechar. Isso cria caminhos de gradiente que não desaparecem tão facilmente.

PROBLEMAS DAS RNN(REDES NEURAIS RECORRENTES)

LSTM (Long Short-Term Memory)

A LSTM, proposta por Hochreiter & Schmidhuber em 1997, foi a primeira e é a mais famosa dessas arquiteturas. Ela introduz uma estrutura de "esteira rolante" chamada **estado da célula (cell state)**.

A LSTM possui três portões principais que controlam a interação com o estado da célula:

- **Forget Gate (Portão de Esquecimento):** Decide qual informação deve ser descartada do estado da célula anterior.
- **Input Gate (Portão de Entrada):** Decide quais novos valores serão armazenados no estado da célula.
- **Output Gate (Portão de Saída):** Decide qual será a saída (próximo estado oculto) com base no estado da célula.

PROBLEMAS DAS RNN(REDES NEURAIS RECORRENTES)

GRU (Gated Recurrent Unit)

A GRU, proposta por Cho et al. em 2014, é uma variação mais recente e simplificada da LSTM. Ela mantém a ideia dos portões, mas com uma arquitetura mais enxuta, combinando o estado da célula e o estado oculto em um único vetor. A GRU possui apenas dois portões:

- **Reset Gate (Portão de Reset):** Decide quanto do estado anterior deve ser esquecido/ignorado ao calcular a nova informação candidata. Se o reset gate for próximo de 0, a ignora o estado anterior, permitindo que a rede comece uma nova memória do zero. É útil para modelar dependências de curto prazo.
- **Update Gate (Portão de Atualização):** Decide quanto do estado anterior deve ser mantido para o novo estado. É uma combinação dos papéis do Forget Gate e do Input Gate da LSTM. Ele controla o fluxo de informação, agindo como um controle de "vazamento" de memória.

PROBLEMAS DAS RNN(REDES NEURAIS RECORRENTES)

Como vimos anteriormente, as LSTMs e GRUs resolveram o problema crítico do gradiente desaparecente. Isso permitiu:

Que os modelos mais profundos temporalmente: Redes capazes de processar sequências de centenas (e eventualmente milhares) de tokens. Além de que o aprendizado de contexto mais longo: Frases inteiras, parágrafos e, finalmente, documentos poderiam ser processados com coerência.

A evolução venho por meio do conceito e **Atenção (Attention)**, introduzido por Bahdanau et al. em 2014 para melhorar modelos de tradução baseados em LSTM.

A LSTM comprimia toda a frase de entrada em um único vetor de contexto no final. Se a frase fosse longa, informação se perdia. Em vez de um único vetor, o modelo podia "olhar para trás" e **pesar a relevância** de todas as palavras da entrada a cada passo da saída.

PROBLEMAS DAS RNN(REDES NEURAIS RECORRENTES)

Em 2017, o artigo **"Attention Is All You Need"** (Vaswani et al.) apresentou a arquitetura **Transformer**, que é a base de todas as LLMs modernas (GPT, BERT, LLaMA, etc.).

O Transformer eliminou a recorrência completamente, assim não há mais LSTMs ou GRUs.

Com o Auto-Atenção (Self-Attention), temos um mecanismo de atenção generalizado para que cada token em uma sequência possa "prestar atenção" a todos os outros tokens diretamente, em paralelo.

Como não há mais sequencialidade, foram adicionadas "codificações posicionais" para que o modelo saiba a ordem das palavras.

As LSTMs, especialmente em tarefas de modelagem de linguagem (ex: próximo caractere/palavra), demonstraram o poder de treinar modelos massivos em grandes corpora de texto.

Exemplo notável: O **ULMFiT** (2018), que era baseado em LSTM, foi um dos primeiros a popularizar a fórmula que as LLMs usam hoje:

- Pré-treinar em um grande corpus genérico (ex: Wikipedia).
- Ajustar fino (fine-tune) para uma tarefa específica.

PROBLEMAS DAS RNN(REDES NEURAIS RECORRENTES)

Sem esse sucesso inicial das LSTMs em modelagem de linguagem, a transição para Transformers em larga escala poderia não ter ocorrido tão rapidamente.

Inspiração para o Mecanismo de Atenção

O mecanismo de atenção foi inicialmente proposto como uma extensão para LSTMs em tradução automática. Foi ao observar que "a atenção funciona muito bem" que se teve a ideia de remover a LSTM. Assim o Transformer nasceu, as LSTMs foram o "incubador" onde a atenção foi testada e validada.

EXERCíCIO

EXERCÍCIO 2

- PARA ESTA ATIVIDADE ESCOLHA DOIS DOS TRÊS MODELOS DE DEEP LEARNING APRESENTADOS DURANTE A SEGUNDA AULA;
- REDES NEURAIS CONVOLUCIONAIS; REDES NEURAIS RECORRENTES ;REDE NEURAIS GENERATIVAS ADVERSARIAIS;
- APÓS ESCOLHER, PROCURE POR ARTIGOS QUE FALEM SOBRE CADA UM DOS TEMAS ESCOLHIDOS;
- FAÇA A LEITURA DOS MESMOS E AVALIE A APLICAÇÃO CRIADO NO ARTIGO; COMPARE OS RESULTADOS OBTIDOS E AS ATIVIDADES NAS QUAIS OS AUTORES REALIZARAM COM MODELO;
- APÓS A SUA ANÁLISE CRIE UMA APRESENTAÇÃO EXPLICANDO OS PONTOS SOLICITADOS;
- O ARTIGO DEVE TER NO MÁXIMO 5 ANOS E APÓS A ESCOLHA A SUA REFERÊNCIA DEVE SER ADICIONADA NO FÓRUM ARTIGOS COMPARATIVOS MACHINE LEARNING (ADICIONADO NO CONTEÚDO DA AULA);
- VERIFIQUE SE O ARTIGO POSTADO NÃO FOI ESCOLHIDO POR OUTRO COLEGA, SERÁ ACEITO SOMENTE O QUE FOI POSTADO PRIMEIRO;
- CASO QUEIRA MODIFICAR O ARTIGO ESCOLHIDO, O ALUNO DEVE ESCOLHER OUTRO QUE NÃO POSTADO E ALTERAR SUA POSTAGEM;

REFERÊNCIAS

BISONG, Ekaba; BISONG, Ekaba. Google colaboratory. **Building machine learning and deep learning models on google cloud platform: a comprehensive guide for beginners**, p. 59-64, 2019.

DE ANDRADE, Amanda Figueiredo et al. A ÉTICA NO USO DA INTELIGÊNCIA ARTIFICIAL E SEUS RISCOS JURÍDICOS. Revista Acadêmica Online, v. 11, n. 56, p. e1400-e1400, 2025.

ISZCZUK, Ana Claudia Duarte et al. Evoluções das tecnologias da indústria 4.0: dificuldades e oportunidades para as micro e pequenas empresas. **Brazilian Journal of Development**, v. 7, n. 5, p. 50614-50637, 2021.

MARCONI, Francesco. Artificial Intelligence Backbone. < https://twitter.com/fpmarconi/status/794208040207740928 >, 2016 Acesso 01/01/2019.

MICHALSKI, Ryszard Stanislaw; CARBONELL, Jaime Guillermo; MITCHELL, Tom M. (Ed.). **Machine learning: An artificial intelligence approach**. Springer Science & Business Media, 2013.

ROSÁRIO, João Maurício. **Automação industrial**. Editora Baraúna, 2012.

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial intelligence: a modern approach**. Malaysia; Pearson Education Limited, 2016.

SANTOS, Gustavo Soares. Novas Tecnologias Aplicadas na Construção Civil: Conceitos da Indústria 4.0. RCT-Revista de Ciência e Tecnologia, v. 8, 2022.

ZHOU, Zhi-Hua. **Machine learning**. Springer nature, 2021.